

Structured Differentiable Optimization for Efficient Decision-Focused Learning in Power Systems

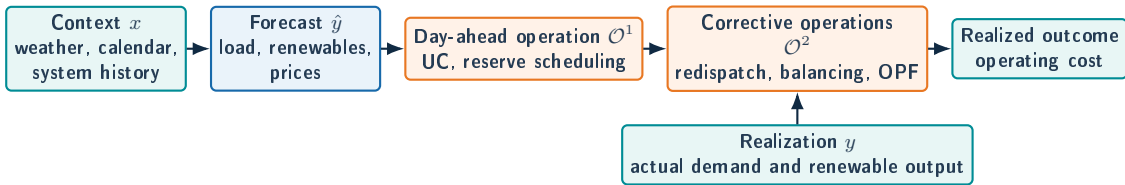
Theory, software, and IEEE 118-bus evidence

Wangkun Xu | Fei Teng

Department of Electrical and Electronic Engineering, Imperial College London

August 7, 2026 | EPICS Bimonthly Meeting

Power-system operations follow a sequential decision chain



General sequential decision model

$$\hat{y} = f_{\theta}(x), \quad z^{1,*} = \mathcal{O}^1(\hat{y}), \quad z^{k,*} = \mathcal{O}^k(z^{k-1,*}; \xi^k), \quad k = 2, \dots, n,$$

$$\text{chain performance} = \ell(z^{1,*}, \dots, z^{n,*}, y, \hat{y}).$$

ξ^k denotes information revealed before stage k , such as realized demand and renewable output y .

Forecast accuracy is not the operational objective

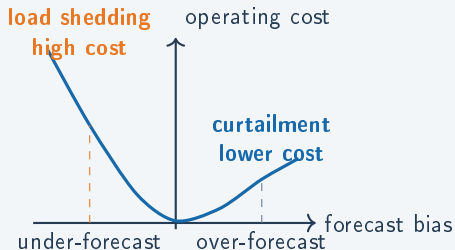
Accuracy-based learning (AbL)

$$\min_{\theta} \frac{1}{2N} \sum_{(x,y)} \|y - f_{\theta}(x)\|_2^2$$

Errors of equal magnitude are penalized equally.

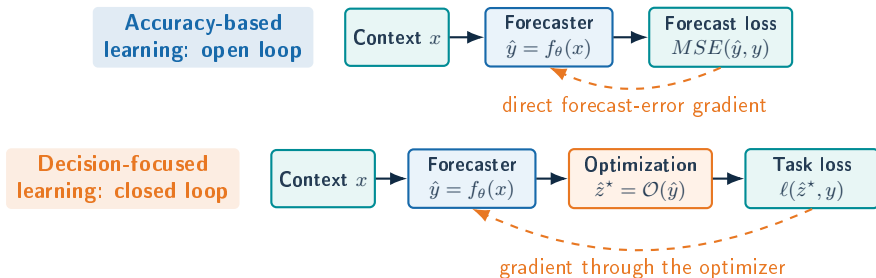
- lower MSE need not imply lower operating cost.
- load-shedding, redispatch, and curtailment costs are asymmetric.

What the operator experiences



Forecasts should be judged through the decisions they induce. However, due to generator limits and network congestion, the mapping from forecast and operational cost is hard to capture.

Decision-focused learning closes the loop



AbL objective

$$\min_{\theta} \frac{1}{N} \sum_{(x,y)} MSE(f_{\theta}(x), y)$$

DfL objective

$$\min_{\theta} \frac{1}{N} \sum_{(x,y)} \ell(\mathcal{O}(\hat{y}), y)$$

Neural network training requires both a *forward pass* for the loss and a *backward pass* for parameter gradients.

Forward: The optimization layer maps forecasts to decisions

Parametric operational optimization

$$\begin{aligned} \mathcal{O}(\hat{y}) : \quad & \hat{z}^*(\hat{y}) \in \arg \min_z f_0(z, \hat{y}) \\ \text{s.t.} \quad & g(z, \hat{y}) = 0, \\ & h(z, \hat{y}) \leq 0 \end{aligned}$$

- $\hat{y}$: forecast-dependent problem parameters;
- z : operational decisions, e.g., dispatch and flows;
- $\hat{z}^*(\hat{y})$: optimizer output used by downstream layers.

Two useful outputs

Solution map

$$\hat{y} \mapsto \hat{z}^*(\hat{y})$$

Value function

$$\hat{y} \mapsto \alpha(\hat{y}) = f_0(\hat{z}^*(\hat{y}), \hat{y})$$

Forward pass

For a given forecast, the solver returns the optimal decision $\hat{z}^*$ or its objective value α .

Backward: why must gradients pass through the optimizer?



backpropagation must cross the optimization layer

Backpropagation: The cost signal follows the decision chain in reverse

$$\underbrace{\nabla_{z^*} \ell}_{\text{cost change from a decision change}} \implies \underbrace{\nabla_{\hat{y}} \mathcal{O}(\hat{y})}_{\text{solution sensitivity to forecast perturbation}} \implies \underbrace{\nabla_{\theta} f_{\theta}(x)}_{\text{forecast gradient}} .$$

What is missing?

A conventional solver returns the optimal decision z^* , but not how sensitive it is to the forecast:

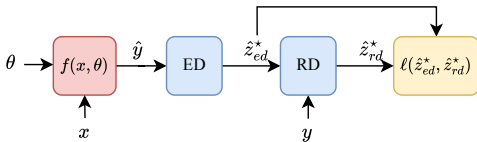
Differentiable optimization provides the solution sensitivity to forecast perturbation, which is crucial for DfL.

Two ED–RD learning formulations

Closed-loop: solution map

$$\begin{aligned}\hat{z}^{ed,*} &= \mathcal{O}^{ed}(\hat{y}), \\ \hat{z}^{rd,*} &= \mathcal{O}^{rd}(\hat{z}^{ed,*}, y).\end{aligned}$$

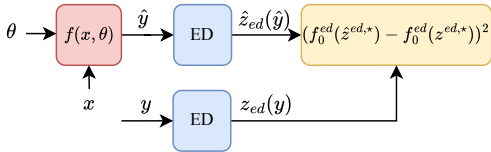
Differentiate the **coupled ED–RD decision chains**.



Counterfactual: value function

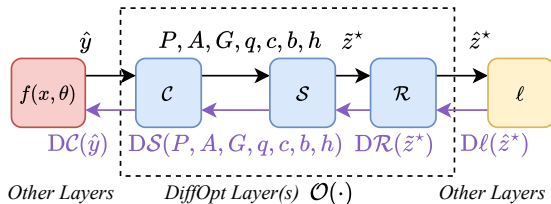
$$\alpha(\hat{y}) = \min_z f_0(z, \hat{y}).$$

Cache the oracle cost and differentiate only the **optimal value**.



Differentiation strategies can be tailored to the application and the desired sensitivity.

Differentiable optimization turns a solver into a layer



1. Canonicalization $\mathcal{C}$

Translate the engineer-facing model into the standard numerical data required by a solver.

2. Solution $\mathcal{S}$

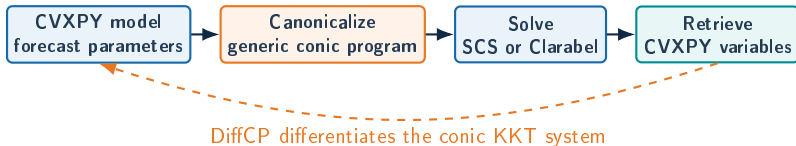
Call a numerical backend to solve the standard problem and return its primal–dual optimum $\tilde{z}^*$.

3. Retrieval $\mathcal{R}$

Map the standard-form solution back to named power-system decisions, such as dispatch, reserve, and flows.

Differentiable optimization has three steps. Forward follows the black arrows; backward must propagate the loss gradient through $\mathcal{R}$, $\mathcal{S}$, and $\mathcal{C}$.

CvxpyLayers implements the layer through conic programs



Why it is widely used for power system DfL

- engineering-friendly CVXPY syntax;
- automatic canonicalization: little optimization knowledge required;
- broad disciplined-conic problem support;
- automatic gradients with respect to model parameters.

Why exact DfL becomes a computational barrier

Training loop

```
for epoch in epochs:
    for batch in data:
         $\hat{y} \leftarrow \text{forecaster}(x)$  in parallel
         $z^* \leftarrow \text{solve } \mathcal{O}(\hat{y})$  in parallel
         $\ell \leftarrow \text{operation-cost}(z^*, y)$ 
         $\ell.\text{backward}()$ 
```

- **Forward**: optimization solve(s) for every sample;
- **Backward**: differentiate the through the optimization.

Where the cost enters in CvxpyLayers

- every sample requires a numerical conic solve;
- backward pass through the optimization is computationally expensive;
- the generic route does not target repeated power system optimization structure.
- the forward solution and backward differentiation are coupled, meaning that only limited number of conic solvers are supported.

We need to solve and differentiate $N_{\text{epochs}} \times N_{\text{samples}}$ number of optimizations

Evidence: power-system DfL remains small-scale

Study	Test system	Forecast	Operation (optimization class)
Donti et al. (2017)	Aggregator (no network)	Load	Redispatch (stochastic LP)
Han et al. (2021)	IEEE 14-bus	Load	Redispatch (stochastic LP)
Vohra et al. (2023)	IEEE 6-bus	Wind, solar	Dispatch/redispatch (LPs)
Zhang et al. (2024)	VPP (no network)	Wind	Dispatch/redispatch (LPs)
Wahdany et al. (2023)	IEEE 6-bus	Wind	Dispatch (LP)
Stratigakos et al. (2025)	Aggregator (no network)	Wind	Redispatch (stochastic LP)
Paredes et al. (2025)	BESS (no network)	Price, etc.	Market clearing (LPs)

What the literature shows

Network-constrained demonstrations use only IEEE 6- or 14-bus systems; the remaining studies omit the network and the scheduling horizon.

The missing evidence is solver-based DfL at realistic network and scheduling scale. We aim to scale up power system DfL with DiffAPQP.

DiffAPQP targets affine-parametric QP

Affine-parametric quadratic program (APQP)

$$\begin{aligned} \min_z \quad & \frac{1}{2} z^\top P z + q(\hat{\mathbf{y}})^\top z + c(\hat{\mathbf{y}}) \\ \text{s.t.} \quad & A z = b(\hat{\mathbf{y}}) \quad (\nu) \\ & G z \leq h(\hat{\mathbf{y}}) \quad (\lambda), \end{aligned}$$

$\hat{\mathbf{y}} := (\hat{\mathbf{y}}_1, \dots, \hat{\mathbf{y}}_{N_p})$ collects the N_p parameter blocks.

$$\begin{aligned} q(\hat{\mathbf{y}}) &= \tilde{q} + \sum_{i=1}^{N_p} Q_i \hat{\mathbf{y}}_i, & b(\hat{\mathbf{y}}) &= \tilde{b} + \sum_{i=1}^{N_p} B_i \hat{\mathbf{y}}_i, \\ h(\hat{\mathbf{y}}) &= \tilde{h} + \sum_{i=1}^{N_p} H_i \hat{\mathbf{y}}_i, & c(\hat{\mathbf{y}}) &= \tilde{c} + \sum_{i=1}^{N_p} d_i^\top \hat{\mathbf{y}}_i. \end{aligned}$$

Fixed across training

$P \succeq 0$, A , G , H_i , Q_i , B_i , d_i
cost curvature, physics, and
network topology, or maps

Power-system APQP examples

Continuous convex formulations of
DC-OPF, ED, RD, market clearing,
bidding, and MPC:

- load and renewable forecasts enter $b(\hat{\mathbf{y}})$ and $h(\hat{\mathbf{y}})$ affinely;
- price forecasts can enter $q(\hat{\mathbf{y}})$ affinely.

DiffAPQP automatic canonicalization

Engineer-facing power system model

$$\begin{aligned} \min_{P_g} \quad & c_g^\top P_g \\ \text{s.t.} \quad & P_{bus} = C_g P_g - C_d P_d, \\ & -P_f^{\max} \leq M P_{bus} \leq P_f^{\max}, \\ & \mathbf{1}^\top P_{bus} = 0, \\ & P_g^{\min} \leq P_g \leq P_g^{\max}. \end{aligned}$$

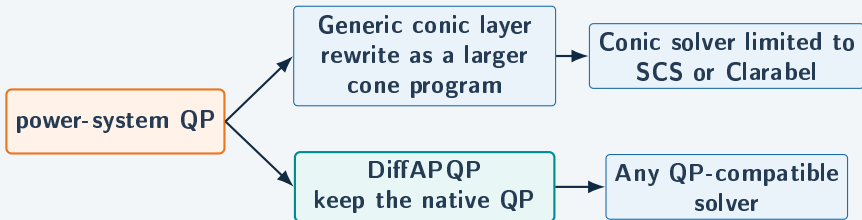
Canonical APQP data

$$\begin{aligned} z &= P_g, & \hat{y} &= P_d, \\ A &= \mathbf{1}^\top C_g, & B &= \mathbf{1}^\top C_d, \\ G &= \begin{bmatrix} M C_g \\ -M C_g \\ I \\ -I \end{bmatrix}, & H &= \begin{bmatrix} M C_d \\ -M C_d \\ 0 \\ 0 \end{bmatrix}. \end{aligned}$$

Solver backend requires standard-form matrices. Manual canonicalization is error-prone. Therefore, DiffAPQP implements automatic canonicalization into a form that is suitable for both solver backends and gradient computation.

DiffAPQP Forward acceleration I: solve QPs as QPs

Two canonicalization routes from the same operational model



Compared to CvxpyLayers, the forward and backward passes of DiffAPQP are separated. DiffAPQP keeps the native QP structure and supports any QP-compatible solver.

Two structures are hiding in plain sight for CvxpyLayers

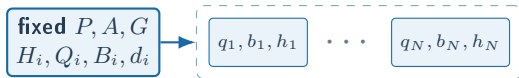
1. Affine parametric structure

Forecasts typically enter through

$$q(\hat{\mathbf{y}}), \quad b(\hat{\mathbf{y}}), \quad h(\hat{\mathbf{y}}), \quad c(\hat{\mathbf{y}})$$

while the quadratic and constraint matrices remain fixed.

Across samples and epochs



different affine data; identical optimization structure: reuse shared structure

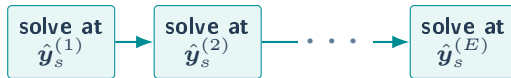
2. Repeated-solve structure

The same dataset sample s returns every epoch:

$$\hat{\mathbf{y}}_s^{(1)}, \hat{\mathbf{y}}_s^{(2)}, \dots, \hat{\mathbf{y}}_s^{(E)}.$$

Consecutive problems are structurally identical and become nearby as training converges.

Across epochs $1, \dots, E$ for sample s



arrows: because $\hat{\mathbf{y}}_s^{(e)}$ values are close, the previous epoch solve can inform the next.

Forward acceleration II: reuse repeated-solve state

1. Solver-data update

$$\begin{aligned} \min_z \quad & \frac{1}{2} z^\top P z + q(\hat{\mathbf{y}})^\top z + c(\hat{\mathbf{y}}) \\ \text{s.t.} \quad & A z = b(\hat{\mathbf{y}}) \quad (\nu) \\ & G z \leq h(\hat{\mathbf{y}}) \quad (\lambda), \end{aligned}$$

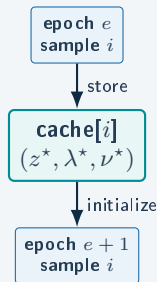
For ADMM-like methods, each iteration solves

$$\underbrace{\begin{bmatrix} P + \rho G^\top G & A^\top \\ A & 0 \end{bmatrix}}_{K \text{ fixed across samples and epochs}} \begin{bmatrix} z^{k+1} \\ w^{k+1} \end{bmatrix} = r(q, b, h, z^k, u^k).$$

K fixed across samples and epochs

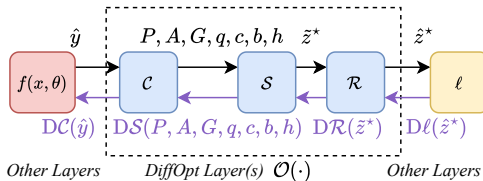
We can compute the inverse (or its factorization) of K once and reuse it for every sample and epoch.

2. Sample-indexed warm start



At epoch $e + 1$, reuse the solution of the same sample i from the previous epoch e ; Benefits grow as forecasts stabilize;

Backward through DiffAPQP: three derivative steps



$$D\mathcal{O}(\hat{y}) = \underbrace{DR}_{\text{retrieve}} \underbrace{DS}_{\text{differentiate the optimum}} \underbrace{DC}_{\text{canonicalize}}.$$

DR: fixed linear map

Select and reshape canonical solution blocks into physical engineering-facing decisions.

DS: difficult step

Determine how the optimal primal–dual solution changes with QP data.

DC: affine map

Propagate forecast gradients through known coefficient matrices.

For the proposed APQP structure, retrieval and canonicalization are analytical; the solver map carries the costly implicit derivative.

Differentiate retrieval $\mathcal{R}$

canonical solution packed into $\tilde{z}^*$
 $[\dots | P_g^* | \theta^* | f^* | r^* | \dots]$

named CVXPY variables
dispatch, angles, flows, reserve

A fixed block-selection matrix

For an original decision variable $\hat{z}_i^*$ stored inside $\tilde{z}^*$,

$$\hat{z}_i^* = \mathcal{R}_i(\tilde{z}^*) = \underbrace{[0 \quad \dots \quad I \quad \dots \quad 0]}_{R_i} \tilde{z}^*, \quad D\mathcal{R}_i = R_i.$$

Forward

Recover the original physical variable layout recorded during compilation.

Backward

Scatter the gradient back to the selected block.

Differentiation on retrieval is analytical and determined by the canonicalization step.

Differentiate canonicalization $\mathcal{C}$

Forward: evaluate a fixed affine template

For $\hat{\mathbf{y}} = (\hat{y}_1, \dots, \hat{y}_{N_p})$,

$$q(\hat{\mathbf{y}}) = q_0 + \sum_i Q_i \hat{y}_i, \quad b(\hat{\mathbf{y}}) = b_0 + \sum_i B_i \hat{y}_i,$$

$$h(\hat{\mathbf{y}}) = h_0 + \sum_i H_i \hat{y}_i, \quad c(\hat{\mathbf{y}}) = c_0 + \sum_i d_i^\top \hat{y}_i.$$

The matrices P , A , G , H_i , Q_i , B_i , and d_i remain fixed.

Power-system interpretation

Load, renewable, reserve, and price blocks enter the right-hand sides or linear objective through their own B_i , H_i , or Q_i .

Backward: choose parameter i

$$D_{\hat{y}_i} \mathcal{C} = \begin{bmatrix} Q_i \\ B_i \\ H_i \\ d_i^\top \end{bmatrix}, \quad i = 1, \dots, N_p.$$

The derivative is directly given by the corresponding coefficient maps.

Reuse

The fixed maps are extracted once and reused in every backward pass.

Differentiate the solver map $\mathcal{S}$

What remains after $\mathcal{C}$ and $\mathcal{R}$?



$$D\mathcal{O} = \underbrace{D\mathcal{R}}_{\text{analytical}} \boxed{D\mathcal{S}} \underbrace{D\mathcal{C}}_{\text{analytical}}.$$

Unlike $\mathcal{C}$ and $\mathcal{R}$, $\mathcal{S}$ is not an explicit algebraic map: a numerical iterative algorithm returns its optimum.

KKT conditions characterize the optimum

$$\begin{aligned} Pz^* + q + A^\top \nu^* + G^\top \lambda^* &= 0, \\ Az^* - b &= 0, \\ Gz^* - h &\leq 0, \quad \lambda^* \geq 0, \\ \lambda^* \odot (Gz^* - h) &= 0. \end{aligned}$$

Compactly,

$$\mathcal{K}(z^*, \nu^*, \lambda^*; \hat{y}) = 0.$$

Differentiate the KKT equilibrium to obtain the solver map sensitivity $D\mathcal{S}$.

Differentiate the solver map $\mathcal{S}$ by the chain rule

The forward computation is the composition

$$\hat{\mathbf{y}} = f_{\theta}(x) = (\hat{y}_1, \dots, \hat{y}_{N_p}), \quad w^{\star} = \mathcal{S}(\mathcal{C}(\hat{\mathbf{y}})), \quad z^{\star} = \mathcal{R}(w^{\star}), \quad L(\theta) = \ell(z^{\star}, y).$$

Therefore, with $w^{\star} = (z^{\star}, \nu^{\star}, \lambda^{\star})$, the training gradient follows directly from the chain rule:

$$D_{\theta} \ell = \sum_{i=1}^{N_p} \underbrace{D\ell}_{v^{\top}} D\mathcal{R} \underbrace{D_{\hat{y}_i} w^{\star}}_{\text{sensitivity to parameter } i} \underbrace{D_{\theta} f_{\theta,i}}_{\text{forecaster}}.$$

PyTorch provides $D\ell$ and $D_{\theta} f_{\theta}$; $D\mathcal{R}$ is analytical.

With $s = Gz^{\star} - h(\hat{\mathbf{y}})$, differentiate the KKT conditions with respect to parameter i gives $D_{\hat{y}_i} w^{\star}$:

$$\underbrace{\begin{bmatrix} P & A^{\top} & G^{\top} \\ A & 0 & 0 \\ \text{diag}(\lambda^{\star})G & 0 & \text{diag}(s) \end{bmatrix}}_{K_a \text{ (full KKT matrix)}} D_{\hat{y}_i} w^{\star} = \underbrace{\begin{bmatrix} -Q_i \\ B_i \\ \text{diag}(\lambda^{\star})H_i \end{bmatrix}}_{D_{a,i}}.$$

Differentiate the solver map $\mathcal{S}$ by the adjoint method

Hence $D_{\hat{y}_i} w^* = K_a^\dagger D_{a,i}$. Substituting this expression into the chain rule yields

$$D_{\hat{y}_i} \ell = \boxed{v^\top K_a^\dagger} D_{a,i}, \quad v = D\mathcal{R}^\top D\ell^\top.$$

We do not explicitly construct $K_a^\dagger$ or form $K_a^\dagger D_{a,i}$ for every parameter block, which is too expensive. Define the **adjoint** by $(s_a^*)^\top = v^\top K_a^\dagger$, which can be solved by a linear system:

$$s_a^* = \arg \min_{s_a} \|K_a^\top s_a - v\|_2^2, \quad D_{\hat{y}_i} \ell^\top = D_{a,i}^\top s_a^*, \quad i = 1, \dots, N_p.$$

The same adjoint s_a^* gives the gradient for every parameter block; only the final multiplication by $D_{a,i}^\top$ changes with i .



Remaining bottleneck

K_a is large, asymmetric, and may be ill-conditioned. This motivates the reduced KKT system.

Differentiation via reduced-KKT system: keep what binds

Identify the active set

When the z^* is obtained from forward pass, we can identify

$$\mathcal{A} = \{j : |G_j z^* - h_j(\hat{\mathbf{y}})| \leq \varepsilon\}.$$

In power-system QPs, many generator and transmission bounds are inactive.



orange: active; gray: removed

This results in an equality-constrained QP, whose KKT differentiation is easier to solve.

Differentiate an equality-constrained QP

$$\underbrace{\begin{bmatrix} P & A^\top & G_{\mathcal{A}}^\top \\ A & 0 & 0 \\ G_{\mathcal{A}} & 0 & 0 \end{bmatrix}}_{K_s \text{ (symmetric)}} \begin{bmatrix} D_{\hat{y}_i} z^* \\ D_{\hat{y}_i} \nu^* \\ D_{\hat{y}_i} \lambda_{\mathcal{A}}^* \end{bmatrix} = \begin{bmatrix} -Q_i \\ B_i \\ H_{i,\mathcal{A}} \end{bmatrix}, \quad i = 1, \dots, N_p$$

- dimension: reduce from $n_d + n_{eq} + n_{in}$ to $n_d + n_{eq} + |\mathcal{A}|$;
- symmetric system enables more efficient solver;
- removes inactive complementarity blocks for numerical stability.

The gain grows when $|\mathcal{A}| \ll n_{in}$, a common regime in network operations.

What is guaranteed by the reduced system?

Sufficient regularity assumptions

- ① forward primal–dual solution satisfies KKT;
- ② strict complementarity;
- ③ P positive definite on the active-constraint null space;
- ④ LICQ for equality and active inequality constraints.

For LPs, a nondegenerate vertex condition replaces the second-order QP condition.

Equivalence result

Under these conditions:

- both full and reduced KKT matrices are nonsingular;
- inactive dual sensitivities are zero;
- primal, equality-dual, and active-dual sensitivities are identical.

$$D_{\hat{y}_i} z^a = D_{\hat{y}_i} z^s,$$

$$D_{\hat{y}_i} \lambda_{\mathcal{A}}^a = D_{\hat{y}_i} \lambda_{\mathcal{A}}^s$$

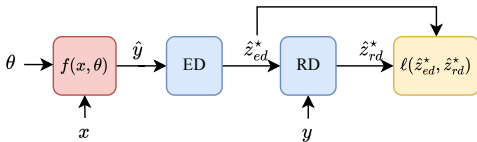
$$D_{\hat{y}_i} \lambda_{\mathcal{I}}^a = 0$$

Reduced KKT is not a heuristic shortcut: at regular points it returns the same derivative.

Two ED–RD learning formulations (again)

Closed-loop: solution map

$$\begin{aligned}\hat{z}^{ed,*} &= \mathcal{O}^{ed}(\hat{y}), \\ \hat{z}^{rd,*} &= \mathcal{O}^{rd}(\hat{z}^{ed,*}, y).\end{aligned}$$

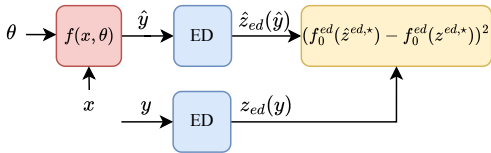


Differentiate the **coupled ED–RD decision chains**.

Counterfactual: value function

$$\alpha(\hat{y}) = \min_z f_0(z, \hat{y}).$$

Cache the oracle cost and differentiate only the **optimal value**.



Differentiation strategies can be tailored to the application and the desired sensitivity.

Differentiate the value function I: start from the APQP

The same multi-parameter APQP

$$\begin{aligned}\alpha(\hat{\mathbf{y}}) = \min_z \quad & \frac{1}{2} z^\top P z + q(\hat{\mathbf{y}})^\top z + c(\hat{\mathbf{y}}) \\ \text{s.t.} \quad & A z = b(\hat{\mathbf{y}}) \quad (\nu), \\ & G z \leq h(\hat{\mathbf{y}}) \quad (\lambda),\end{aligned}$$

where $\hat{\mathbf{y}} = (\hat{y}_1, \dots, \hat{y}_{N_p})$ and each parameter block enters through its maps Q_i , B_i , H_i , and d_i .

Solution-map layer

Returns decisions z^* ; backward requires $D_{\hat{y}_i} z^*$ from a KKT linear system: full or reduced.

Value-function layer

When the training loss only depends on the optimal value, a “free” differentiation strategy is available.

When the learning loss uses only α , the envelope theorem avoids differentiating the optimizer solution z^* .

Differentiate the value-function II: duals carry the sensitivity

From the full chain rule to the envelope result

Define the Lagrangian as: $\mathcal{L}(z, \nu, \lambda; \hat{\mathbf{y}}) = f_0(z, \hat{\mathbf{y}}) + \nu^\top (Az - b(\hat{\mathbf{y}})) + \lambda^\top (Gz - h(\hat{\mathbf{y}}))$.
Then, the envelope theorem states that:

$$\underbrace{D_{\hat{y}_i} \alpha = \partial_{\hat{y}_i} f + \nabla_z f^\top D_{\hat{y}_i} z^*}_{\text{direct effect} + \text{optimizer response}} \xrightarrow[\text{feasibility and complementarity}]{\text{KKT stationarity}} \boxed{D_{\hat{y}_i} \alpha = \partial_{\hat{y}_i} \mathcal{L}}.$$

Apply it to APQP parameter i

$$\boxed{D_{\hat{y}_i} \alpha = z^{*\top} Q_i + d_i^\top - \nu^{*\top} B_i - \lambda^{*\top} H_i}$$

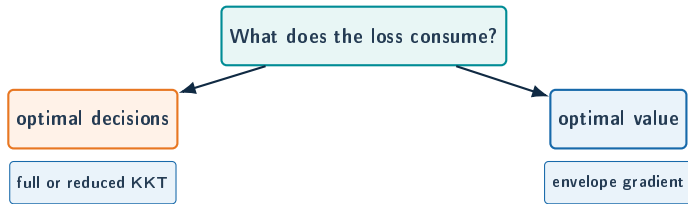
- z^* : optimal system schedule;
- ν^* : marginal value of power balance: the familiar LMP;
- λ^* : shadow prices of binding operating limits.

Computational implication

- All terms come from the forward solve and affine maps;
- Only matrix–vector products are required;
- No full or reduced KKT linear system solve is required.

Three backward paths, chosen by the learning objective

	Full KKT	Reduced KKT	Value function
Layer output	decisions z^*	decisions z^*	scalar α
Backward computation	solve linear system with asymmetric K_a	solve linear system with symmetric K_s	compute matrix–vector products
Linear solver	LSQR	MINRES	none
System size	$n_d + n_{eq} + n_{in}$	$n_d + n_{eq} + \mathcal{A} $	–
Power-system use	closed-loop ED–RD	closed-loop ED–RD	counterfactual ED value



DiffAPQP in one slide

Methodology

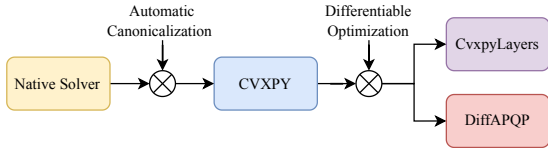
- automatic canonicalization on engineering-facing models;
- warm starts and solver-data updates for repeated-solves.

Theory

- full-KKT solution-map differentiation;
- equivalent reduced active-set KKT system;
- adjoint-free value-function gradient.

Practice

- forward/backward separation and flexible solver choice;
- PyTorch-compatible batched APIs;
- open-source package.



Package usage: from CVXPY model to PyTorch layer

PyTorch-compatible API. Install with `pip install diffapqp`.

```
from diffapqp import ValueFuncLayer
layer = ValueFuncLayer(problem, reduced_kkt=True)

load, solar, wind = forecaster(x)
params = {"load": load, "solar": solar, "wind": wind}
value, *_ = layer(
    params, idx_list=sample_ids, solver="OSQP",
    warm_start=True, update=True
)
loss = ((value - oracle_value) / 1e6).square().mean()
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

When do warm start and solver update activate?

```
layer(params, idx_list=sample_ids, warm_start=True, update=True)
```

Warm start

Activated when

- `warm_start=True`;
- The data sample ID must be provided.
- `cache[sample_id]` contains a previous solver iterate for the same sample.

Update

Activated when

- `update=True`.
- First call: build the workspace.
- Later calls: replace q, b, h while retaining the fixed structure.



DiffAPQP and CvxpyLayers: different design points

Capability	DiffAPQP	CvxpyLayers
Program class	affine-parametric QP/LP	disciplined convex programs
Automatic canonicalization	✓	✓
Forward backends	all compatible continuous QP/conic solvers	only DiffCP-supported backends
Explicit warm start	✓ backend dependent	✗
Solver-data update	✓ backend dependent	✗
Parallel batch solve	✓	✓
Reduced-KKT gradient	✓	✗
Direct value-function gradient	✓	✗
Native named parameters	✓	✓
Native named decision variables	✓	✗

Case study:DfL on IEEE 118-bus system with 24-hour ED–RD chain

IEEE 118-bus system

buses	118
generators	34
transmission branches	186
loads	99
solar / wind plants	10 / 10
horizon	24 h

- Power balance and network constraints;
- generator capacity and ramping;
- curtailment, shedding, and redispatch.

Optimization dimensions

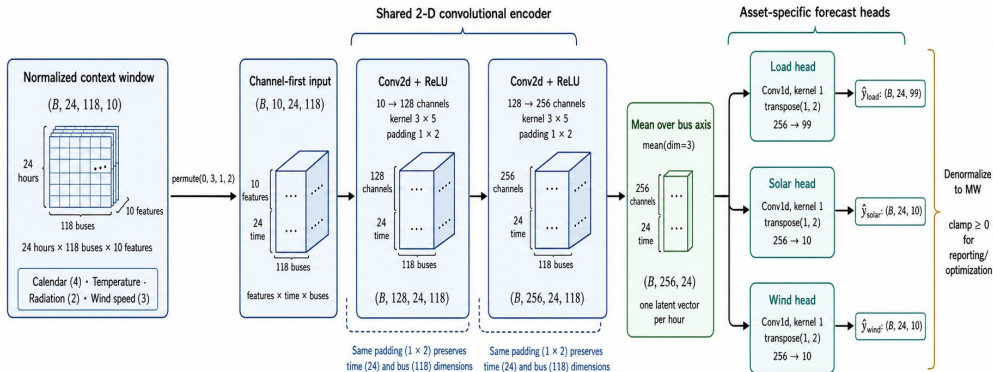
	ED	RD
scalar variables	8,952	10,584
equalities	4,488	4,488
inequalities	19,536	24,432

Data

One year of hourly nodal weather, load, solar, and wind data; preprocessing and testbed construction use GridForge.

118-bus case study with coupled 24-hour ED–RD chain has not been seen in DfL literature.

Shared neural forecaster for load, solar, and wind



B : batch size

$$X \in \mathbb{R}^{B \times 24 \times 118 \times 10}$$

2-D kernels preserve time and bus dimensions

541,687 trainable parameters

Matched experimental protocol

Data and training

- AbL pretraining;
- DfL fine tuning under different settings with CvxpyLayers and DiffAPQP;

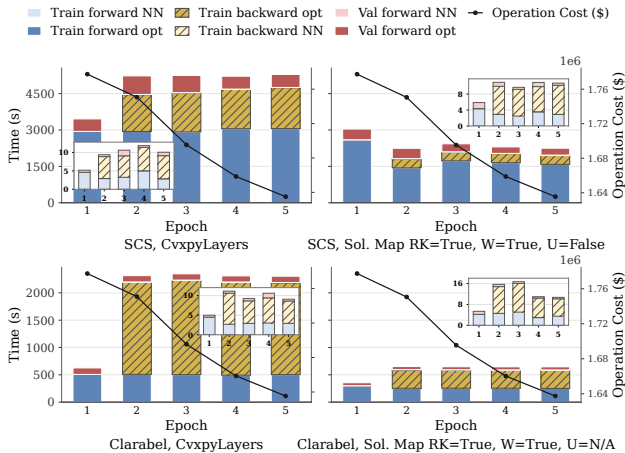
Fairness and hardware

- same random seed;
- matched solver arguments for shared backends;
- Ubuntu 20.04.6 LTS (also partially test on MacOS);
- dual Xeon Gold 6230R, 256 GiB RAM;
- affinity capped at 32 logical CPUs.

Timing definition

Reported runtime is the mean end-to-end epoch time over update epochs 2–5, including training and validation.

Closed-loop result: coupled solution map layers



How to read

Bars decompose the epoch time; black curves track operating cost. Similar cost trajectories indicate comparable learning behavior.

Mean end-to-end epoch speedup: $2.27\times$ with SCS and $3.58\times$ with Clarabel.

Reading the closed-loop speedups

Backend	Method / setting	Epoch	Opt. forward (min)	Opt. backward (min)
SCS	CvxpyLayers	87.43	49.81	26.88
SCS	DiffAPQP, RK+WS	38.44	26.63	6.21
Clarabel	CvxpyLayers	38.65	8.32	28.27
Clarabel	DiffAPQP, RK+WS	10.79	4.17	5.47

SCS

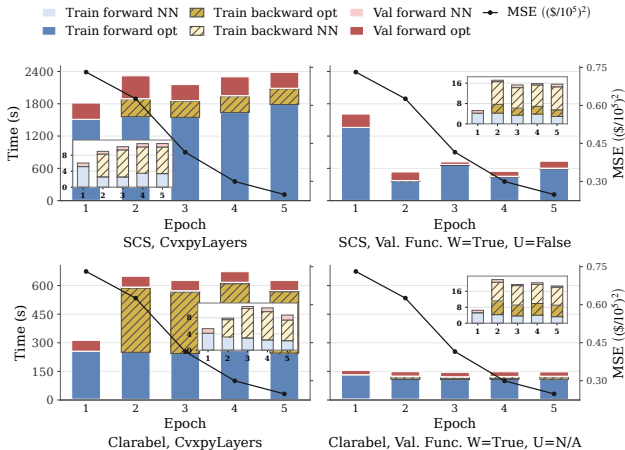
Native APQP + warm start gives $1.87\times$ forward acceleration; reduced KKT gives $4.33\times$ backward acceleration.

Clarabel

Forward and backward components improve by $1.99\times$ and $5.17\times$, respectively.

RK: reduced KKT; WS: warm start. Times are means over epochs 2–5.

Counterfactual result: value-function layer



How to read

Bars decompose the epoch time; black curves track counterfactual MSE. Similar MSE trajectories indicate comparable learning behavior.

Mean end-to-end epoch speedup: $3.62\times$ with SCS and $4.38\times$ with Clarabel.

Why counterfactual differentiation is especially fast

Backend	Method / setting	Epoch	Opt. forward (min)	Opt. backward (min)
SCS	CvxpyLayers	38.16	27.16	5.144
SCS	DiffAPQP value, WS	10.53	8.51	0.049
Clarabel	CvxpyLayers	10.73	4.11	5.553
Clarabel	DiffAPQP value, WS	2.45	1.73	0.099

CvxpyLayers

Return ED decisions, reconstruct objective, and differentiate through the complete conic solution map.

DiffAPQP

Return the optimal ED value and evaluate its gradient directly from the forward primal–dual solution.

The optimization backward pass falls significantly because there is no linear system solve.

Warm start and update are solver and problem dependent

Closed-loop backend	CS	WS	UP	WU	Best observation
SCS	58.57	38.44	60.68	47.65	WS 1.52 $\times$ vs CS
OSQP	33.57	22.04	26.61	18.96	WU 1.77 $\times$ vs CS
QPALM	18.08	10.92	17.66	16.00	WS 1.66 $\times$ vs CS

unit: minute.

Warm start

Most consistent benefit: reuse the same sample's previous solution.

Data update

Helps OSQP here, but not SCS or QPALM when combined with WS.

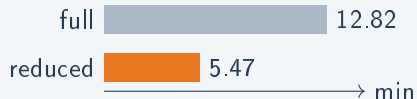
Best backend

Gurobi closed-loop: 9.88 min;
QPALM counterfactual: 1.68 min.

Expose both mechanisms, benchmark them, and report the exact configuration.

Reduced KKT and memory: structure changes the resource profile

Isolating reduced KKT with Clarabel



Optimization backward speedup: 2.34 $\times$.

Peak incremental memory (GiB)

Setting	Cvxpy	DiffAPQP	Red.
Closed SCS	59.53	26.74	55.1%
Closed Clarabel	56.46	28.56	49.4%
Counter SCS	38.68	11.94	69.1%
Counter Clarabel	34.46	12.89	62.6%

The gains come from smaller structured derivative state, not from using more CPU cores.

Computational gains preserve decision quality

Matched-backend outcomes

- closed-loop SCS/Clarabel test costs differ from CvxpyLayers by at most 0.2%;
- counterfactual test-MSE differences are 0.022% and 0.003%;
- learning trajectories remain closely aligned.

Cross-backend maxima

3.91×

closed-loop: Gurobi DiffAPQP vs fastest
CvxpyLayers

6.39×

counterfactual: QPALM DiffAPQP vs fastest
CvxpyLayers

DiffAPQP changes how the optimization is solved and differentiated, without materially changing the learned operating outcome.

Reflections

Structure

Power-system QPs are affine parametric and repeatedly solved. Preserving this structure avoids generic conic overhead from CvxpyLayers.

Differentiation

Active constraints define a smaller equivalent KKT system; and value-only losses need no adjoint system.

Practice

Solver choice, warm start, and update must be evaluated jointly rather than treated as universally beneficial.



The central contribution is a complete path from modeling language to efficient forward and backward passes.

Scope, limitations, and next steps

Current scope

- continuous convex APQPs;
- exact solver-based DfL;
- CPU-parallel forward and sparse implicit backward passes.

Important caveats

Active-set differentiation depends on regularity and tolerance; warm-start/update benefits are backend and problem dependent.

Research directions

- GPU-aware forward and backward adjoint solvers;
- extensions beyond continuous convex APQP structure: mixed-integer or non-convex settings;
- gradient with respect to the dual variables: e.g., when LMP is used as a control signal in demand response.

Scalability is now sufficient for a 118-bus, 24-hour exact DfL demonstration—but solver-aware design remains essential.

Thank you

Questions and discussion

The screenshot shows the DiffAPQP documentation website. The header is dark blue with the DiffAPQP logo, a search bar, and the repository name 'xuwkk/diffapq'. The left sidebar lists navigation links: DiffAPQP, Home, Installation, Solvers, API Overview, Theory, Overview, Decision-Focused Learning, APQP and Canonicalization, Efficient Forward Pass, Solution-Map Differentiation, Value-Function Differentiation, Numerical Considerations, Development Plan, Implementation, Solution-Map Layer, and Value-Function Layer. The main content area has a title 'DiffAPQP' and a paragraph describing its capabilities. A highlighted box titled 'Model directly in CVXPY' explains how to use the package. A 'Table of contents' sidebar on the right lists: Table of contents, Installation, Quick start, Choose a layer, Core capabilities, Current scope, and Continue reading. The 'Installation' section is expanded, showing the command to install the package from PyPI.

DiffAPQP

DiffAPQP provides differentiable optimization layers for continuous affine-parametric quadratic and linear programs. It connects models written in CVXPY to PyTorch, supports batched forward solves through multiple solver backends, and differentiates either optimal decisions or optimal objective values.

Model directly in CVXPY

Write the optimization problem in its natural engineering form and pass the original `cvxpy.Problem` to DiffAPQP. The package validates, canonicalizes, and extracts the compatible APQP structure internally; users do not need to derive standard-form matrices manually.

Installation

Install the core package from PyPI:

```
python -m pip install diffapqp
```

Online documentation and tutorials

Preprint

arXiv:2608.04189

Package and source

PyPI: `diffapqp`

GitHub: `xuwkk/diffapqp`

Documentation and experiments

Online documentation

Power-system experiments

Backup: precise active-set tolerance

The numerical active set is identified using

$$\mathcal{A} = \{j : |G_j z^* - h_j| \leq \max(\varepsilon_{abs}, \varepsilon_{rel}(1 + |h_j|))\}.$$

Too small

Numerically binding constraints may be excluded, invalidating the local active-set model.

Too large

Nearly active or linearly dependent constraints may enter K_s and make it singular or poorly conditioned.

The active-set tolerance should exceed forward feasibility error, but remain selective.

Backup: solver and adjoint settings

Solver	Algorithm	Main settings
OSQP	ADMM/operator splitting	$\varepsilon_{abs} = \varepsilon_{rel} = 10^{-4}$, max. 10,000 iterations
SCS	operator splitting	$\varepsilon_{abs} = \varepsilon_{rel} = 10^{-5}$, max. 100,000 iterations
QPALM	augmented Lagrangian	$\varepsilon_{abs} = \varepsilon_{rel} = 10^{-6}$, max. 10,000 iterations
Gurobi	automatically selected QP algorithm	one thread; feasibility/optimalty tolerances 10^{-8}
Clarabel	interior point	absolute/relative gap tolerances 10^{-8} , max. 10,000 iterations
Full KKT	LSQR	$atol = btol = 10^{-6}$, iteration limit 5,000
Reduced KKT	MINRES	relative tolerance 10^{-9}

Matched solver backends use identical forward tolerances in DiffAPQP and CvxpyLayers.

Backup: closed-loop timing—shared backends

Method	Solver	RK	W	U	Cores	Memory (GiB)	Fwd. (min)	Epoch (min)
CvxpyLayers	SCS	–	–	–	14.65	59.53	49.81	87.43
DiffAPQP	SCS	T	F	F	17.75	26.55	43.54	58.57
DiffAPQP	SCS	T	F	T	17.66	26.77	45.73	60.68
DiffAPQP	SCS	T	T	F	14.08	26.74	26.63	38.44
DiffAPQP	SCS	T	T	T	13.81	26.78	35.22	47.65
CvxpyLayers	Clarabel	–	–	–	17.55	56.46	8.32	38.65
DiffAPQP	Clarabel	F	T	N/A	23.80	28.51	4.22	18.22
DiffAPQP	Clarabel	T	F	N/A	21.74	28.50	5.30	12.53
DiffAPQP	Clarabel	T	T	N/A	20.73	28.56	4.17	10.79

Backup: closed-loop timing—QP backends

Method	Solver	RK	W	U	Cores	Memory (GiB)	Fwd. (min)	Epoch (min)
DiffAPQP	OSQP	T	F	F	19.43	28.23	22.30	33.57
DiffAPQP	OSQP	T	F	T	17.57	28.23	16.01	26.61
DiffAPQP	OSQP	T	T	F	17.46	28.25	12.59	22.04
DiffAPQP	OSQP	T	T	T	15.68	28.16	10.44	18.96
DiffAPQP	QPALM	T	F	F	22.98	28.76	9.81	18.08
DiffAPQP	QPALM	T	F	T	22.20	28.77	9.41	17.66
DiffAPQP	QPALM	T	T	F	17.54	28.74	4.16	10.92
DiffAPQP	QPALM	T	T	T	20.96	28.73	7.98	16.00
DiffAPQP	Gurobi	T	F	N/A	13.54	27.23	3.29	9.88
DiffAPQP	Gurobi	T	T	N/A	12.28	27.25	3.94	10.61

Backup: counterfactual—shared backends

Method	Solver	W	U	Cores	Memory (GiB)	Fwd. (min)	Epoch (min)
CvxpyLayers	SCS	–	–	15.87	38.68	27.16	38.16
DiffAPQP	SCS	F	F	19.82	11.93	23.23	28.16
DiffAPQP	SCS	F	T	20.82	11.96	22.06	26.46
DiffAPQP	SCS	T	F	10.58	11.94	8.51	10.53
DiffAPQP	SCS	T	T	9.04	11.94	9.43	11.80
CvxpyLayers	Clarabel	–	–	17.26	34.46	4.11	10.73
DiffAPQP	Clarabel	F	N/A	22.60	12.86	2.33	3.13
DiffAPQP	Clarabel	T	N/A	24.47	12.89	1.73	2.45

Backup: counterfactual—QP backends

Method	Solver	W	U	Cores	Memory (GiB)	Fwd. (min)	Epoch (min)
DiffAPQP	OSQP	F	F	21.24	12.53	7.93	9.82
DiffAPQP	OSQP	F	T	19.90	12.51	8.44	9.97
DiffAPQP	OSQP	T	F	11.69	12.50	1.84	2.63
DiffAPQP	OSQP	T	T	13.46	12.49	2.09	2.80
DiffAPQP	QPALM	F	F	22.45	13.45	3.63	4.71
DiffAPQP	QPALM	F	T	21.36	13.44	3.69	4.82
DiffAPQP	QPALM	T	F	13.21	13.45	1.11	1.68
DiffAPQP	QPALM	T	T	17.35	13.50	2.25	3.01
DiffAPQP	Gurobi	F	N/A	9.36	12.41	1.51	2.11
DiffAPQP	Gurobi	T	N/A	7.98	12.39	1.81	2.46

Backup: what exactly is timed?

Per epoch

- training NN forward;
- training optimization forward;
- optimization backward;
- NN backward;
- validation NN + optimization forward.

Comparison scope

- average over update epochs 2–5;
- first-call setup remains in epoch 1;
- final test evaluation excluded from epoch speedup;
- wall-clock components use the same instrumentation.

$$\text{speedup} = \frac{\text{CvxpyLayers mean epoch time}}{\text{DiffAPQP mean epoch time}}.$$

For example, closed-loop SCS: $87.43/38.44 = 2.27\times$.

Selected references



A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, and J. Z. Kolter, “Differentiable convex optimization layers,” *NeurIPS*, 2019.



S. Diamond and S. Boyd, “CVXPY: A Python-embedded modeling language for convex optimization,” *JMLR*, 2016.



S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004.



J. Nocedal and S. Wright, *Numerical Optimization*, Springer, 2006.



B. Stellato et al., “OSQP: An operator splitting solver for quadratic programs,” *Mathematical Programming Computation*, 2020.



B. O’Donoghue, “Operator splitting for a homogeneous embedding of the linear complementarity problem,” *SIAM J. Optim.*, 2021.



P. Goulart and Y. Chen, “Clarabel: An interior-point solver for conic programs,” 2024.